초안. 표 2에서 ‡로 표시한 numpy 측정값은 측정 결함(4.5절)의 영향을 받았다. 하네스는 수정하였으며, 전체를 다시 측정한 뒤 이 수치와 그에 기대는 문장(요약, 4.3절, 4.4절)을 갱신한다.

프로그래밍 언어 선택과 언어 특성이 LLM 생성 HPC 코드의 정확성·성능·안전성에 미치는 영향 — 실증 분석

정유성, 허선영†
경희대학교 컴퓨터공학부
e-mail: henry7007@khu.ac.kr

Effects of Programming Language Choice and Language Features on the Correctness, Performance, and Safety of LLM-Generated HPC Code: An Empirical Analysis

Yuseong Jung, Seonyeong Heo†
School of Computing, Kyung Hee University

요 약

대규모 언어 모델(LLM)이 생성한 고성능 계산(HPC) 코드는 결과가 맞는 것만으로는 평가할 수 없다. 루프 순서, 벡터화, 병렬화와 같은 성능 결정에 따라 실행 시간이 크게 달라지기 때문이다. 본 연구는 “같은 과제에서 나타나는 언어 조건 간 성능 격차는 언어의 추상화 수준 자체가 아니라, 성능 결정을 LLM이 직접 내려야 하는지 컴파일러·라이브러리에 위임할 수 있는지로 설명된다”는 가설을 검증하는 것을 목표로 한다. 이를 위해 코드 생성부터 빌드, 격리 실행, 정확성 판정, 오류 분류, 시간 측정까지를 자동화하고 측정 규약을 코드로 강제하는 평가 하네스를 구현하였다. 중간 결과로, 난이도를 세 수준으로 나눈 12개 수치 과제를 세 언어 조건(NumPy, 순차 C, C+OpenMP)에서 5회씩 생성한 180개 응답을 평가하였다. 첫 시도 통과 수는 조건별로 55/60, 55/60, 53/60으로 비슷하였으나 실패의 원인은 조건마다 달랐다. NumPy의 실패 5건은 모두 라이브러리에 위임하기 위한 식 재배열에서, C의 알고리즘·메모리 오류 8건은 모두 블록화나 방향 최적화처럼 모델이 택한 최적화 구조에서 나왔다. 속도를 같은 과제 안에서 비교하면 C+OpenMP 구현은 NumPy 구현의 0.93~1.41배(네 과제) 또는 6.0~18.5배(일곱 과제)였고, 나머지 한 과제는 NumPy 구현이 모두 실패하였다. 앞의 네 과제는 NumPy 구현이 연산의 대부분을 BLAS와 SIMD 정렬 루틴에 위임한 과제이다. 다만 이 구분은 결과를 확인한 뒤에 세운 설명이며, 모델 1종과 언어 조건 3개에 한정된 예비 결과이다. 남은 기간에 언어 조건 확장, 수정 루프, 안전성 분석을 수행한다.

1. 서 론

대규모 언어 모델(LLM)은 범용 프로그래밍 과제에서 높은 정확도로 코드를 생성하게 되었고 [1], [2], 과학 계산과 고성능 계산(HPC) 분야에서도 병렬 코드를 생성하려는 시도가 이어지고 있다 [3]–[6]. 그러나 HPC 코드는 계산 결과가 맞는 것만으로 충분하지 않다. 같은 결과를 내는 두 구현의 실행 시간이 루프 순서, 타일링, 벡터화, 병렬화, 메모리 배치와 같은 성능 결정에 따라 크게 달라지기 때문이다. 예컨대 행렬 곱은 캐시 블록화와 패킹의 유무만으로도 성능이 크게 달라진다 [7].

프로그래밍 언어는 이러한 성능 결정을 누가 내리는지를 정한다. NumPy [8]에서는 배열 연산 하나가 미리 최적화된 라이브러리 루틴으로 실행되므로 작성자가 내릴 결정이 적다. Halide [9]는 알고리즘과 스케줄을 분리하여 스케줄 결정을 자동 탐색에 위임할 수 있다. Numba [10]는 Python으로 쓴 반복문을 기계어로 컴파일하므로, 언어는 Python에 머물면서 루프 구조는 작성자가 정한다. C와 OpenMP에서는 루프 구조와 스레드 분할, 누적 변수의 정밀도까지 작성자, 곧 LLM이 직접 정해야 한다. 따라서 생성 대상 언어의 선택은 문법의 차이를 넘어, LLM이 직접 내려야 하는 성능 결정의 양을 바꾸는 실험 변수가 된다.

본 연구의 가설은 다음과 같다. “같은 과제에서 나타나는 언어 조건 간 성능 격차는 언어의 추상화 수준 자체가 아니라, 성능 결정을 LLM이 직접 내려야 하는지 컴파일러·라이브러리에 위임할 수 있는지로 설명된다.” 본 보고서에서는 이 가설을 두 가지 예측으로 구체화한다. (P1) 연산 전체를 최적화된 루틴에 위임할 수 있는 과제에서는 조건 간 속도 차이가 작다. (P2) 그러한 루틴이 없는 과제에서는 성능 결정을 코드로 직접 표현할 수 있는 조건이 더 빠르다. 이를 확인하기 위해 같은 수치 과제를 성능 결정 부담이 서로 다른 언어 조건에서 생성하고, 정확성과 성능을 같은 규약으로 측정한다(안전성 측정은 남은 과제이다).

본 중간보고서의 내용은 다음과 같다. 첫째, 생성부터 빌드, 격리 실행, 정확성 판정, 오류 분류, 시간 측정까지를 자동화하고 측정 규약을 코드로 강제하는 평가 하네스를 구현하였다(3장). 둘째, 12개 과제를 세 언어 조건에서 5회씩 생성한 180개 응답의 예비 결과를 보고한다(4장). 셋째, 이 결과로 가설에 대해 주장할 수 있는 범위와 그 한계를 구분하고 남은 계획을 제시한다(4.5절, 5장).

2. 관련 연구

코드 생성 평가. HumanEval [1]은 단위 테스트 통과 여부로 생성 코드의 기능적 정확성을 평가하는 방식을 정착시켰고,

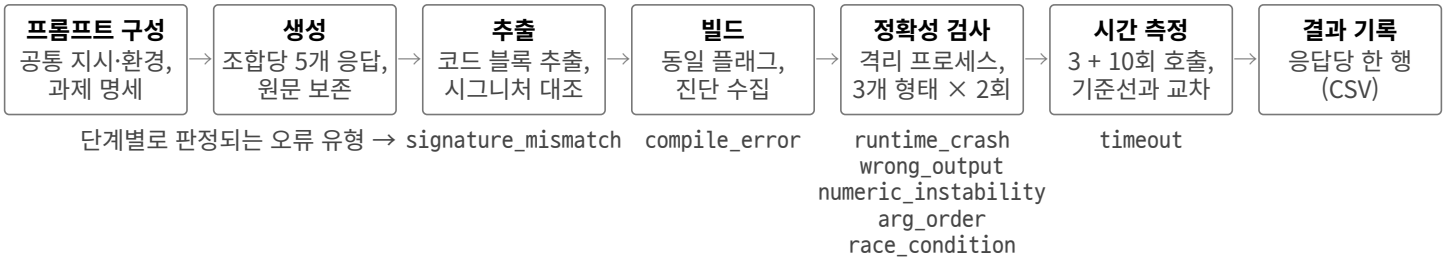


그림 1 평가 하네스의 흐름과 단계별로 판정되는 오류 유형. 사람이 개입하는 단계는 없으며, 어느 단계에서든 실패하면 오류 유형을 기록하고 그 응답의 평가를 끝낸다. 정확성 검사를 통과한 구현만 시간을 측정한다. `timeout`은 정확성 검사에서도 판정되며, 아홉 번째 유형인 `memory_error`는 새니타이저를 연동한 뒤에 판정한다.

MultiPL-E [2]는 이를 18개 언어로 확장하여 언어에 따라 정확도가 달라짐을 보였다. 이들은 범용 프로그래밍 문제의 정확성만을 측정한다.

HPC·병렬 코드 생성. Godoy 등 [4]은 AXPY, GEMM, SpMV 등 6개 커널을 C++, Fortran, Python, Julia의 여러 병렬 프로그래밍 모델로 생성하게 하여 제안된 코드의 적절성을 채점하였고, 후속 연구 [5]는 같은 틀에서 Llama-2와 GPT-3를 비교하였다. ParEval [3]은 12개 문제 유형과 7개 실행 모델에 걸친 420개 프롬프트로 병렬 코드 생성을 평가하고 생성 코드의 속도 향상을 재는 지표를 제안하였다. Diehl 등 [6]은 세 개의 과학 계산 프로그램을 9개 언어로 생성하게 하여 컴파일·실행 여부와 성능을 비교하였으며, 병렬 코드를 올바르게 생성하기가 특히 어렵다고 보고하였다.

생성 코드의 효율. PIE [11]는 사람이 작성한 성능 개선 편집으로 모델의 코드 최적화 능력을 평가하고, EffiBench-X [12]는 6개 언어에서 생성 코드의 실행 효율을 사람의 풀이와 비교한다. 두 연구의 과제는 경쟁 프로그래밍 문제이며 벡터화·병렬화 같은 HPC의 성능 결정은 다루지 않는다. 한편 생성 코드의 안전성은 주로 보안 취약점의 관점에서 보고되었다 [13].

본 연구의 위치. 기존 연구 가운데에는 정확성뿐 아니라 속도를 측정한 것도 있으나, 언어 조건을 성능 결정의 위임 정도에 따라 배열하여 비교한 연구는 찾지 못하였다. 본 연구는 세 가지 점에서 구별된다. 첫째, 언어 조건을 "성능 결정을 누가 내리는가"라는 하나의 축으로 배열한다. 둘째, 같은 과제에 대해 정확성과 기준선 대비 속도를 고정된 측정 규약으로 함께 측정한다. 셋째, 실패를 사전에 정의한 분류표로 분류한다.

3. 연구 설계와 평가 하네스

3.1 변수와 언어 조건

독립 변수는 언어 조건이다. 성능 결정 부담이 작은 쪽부터 `numpy`, `numba`, `halide_auto`(알고리즘만 작성하고 스케줄은 자동), `halide_manual`(스케줄도 작성), C의 순서로 배열하며, 본 보고서는 양 끝에 해당하는 `numpy`와 C의 두 조건(`c_seq`, `c_openmp`)을 다룬다. `numpy`는 Python 표준 라이브러리와 NumPy만 쓸 수 있고, `c_seq`는 C 표준 라이브러리만 쓰는 순차 코드이며, `c_openmp`는 여기에 OpenMP를 허용한다. 언어 조건이 결정의 주체를 완전히 고정하지는 않으므로(예: NumPy 안에서의 스레드 사용), 실제로 누가 결정을 내렸는지는 생성 코드에서 따로 관찰한다(4.4절).

종속 변수는 정확성(통과 여부와 오류 유형), 성능(실행 시간의 중앙값과 속도비), 비용(토큰 수)이다. 속도비는 기준선의 실행 시간을 생성 코드의 실행 시간으로 나눈 값으로, 1보다 크면 기준선보다 빠르다. 안전성(새니타이저와 린트의 탐지 결과)은 기록 항목만 정의해 두었고 아직 측정하지 않았다.

표 1 과제 집합. 가로선으로 나눈 그룹이 위에서부터 난이도 수준 1, 2, 3이다. *는 기준선이 NumPy 라이브러리 호출인 과제이며, 나머지의 기준선은 C+OpenMP 참조 구현이다.

과제	성능 측정 크기	과제별로 상정한 성능 결정
`gemm`*	1024^3	루프 순서, 타일링, 벡터화
`jacobi2d`	4096^2, 20스텝	버퍼 교환, 경계, 병렬 루프
`spmv_csr`	10^6행	행 분할, 간접 참조
`dot`*	2^{25}	리덕션, 누적 정밀도
`histogram`	2^{25}, 구간 1024	스레드별 히스토그램
`scan`	2^{25}	2-pass 병렬 누적 합
`nbody`	16384	자료 배치, 역제곱근
`sort`*	10^7	알고리즘 선택
`cholesky`*	2048^2	블록화, 병렬 루프 판별
`floyd_warshall`	1024^2	반복 간 의존성, 블록화
`bfs`	정점 2^{21}	프론티어 병렬화, 원자 연산
`fft`*	2^{22}	비트 반전, 회전 인자 정밀도

3.2 과제 집합

과제는 표 1의 12개 수치 커널이다. 선정 기준은 (1) 본 보고서의 세 조건 모두에서 구현할 수 있을 것, (2) 명세만으로 생성할 수 있는 크기(함수 1~3개)일 것, (3) 정답을 자동으로 검증할 수 있을 것, (4) 병목의 성격이 다양할 것, (5) 난이도가 단계적으로 높아질 것이다. 수준 1은 기본 커널, 수준 2는 병렬화나 알고리즘 재구성이 자명하지 않은 커널, 수준 3은 반복 간 의존성이나 불규칙 접근이 있는 커널이다. 과제는 PolyBench [14], NAS Parallel Benchmarks [15], GAP [16] 등 널리 쓰이는 벤치마크가 다루는 계산 유형에서 선정하였다.

각 과제는 영어로 쓴 자연어 명세(수학적 정의, 입출력 계약, 크기, 허용오차), 언어별 함수 시그니처, 배정밀도 참조 구현, 시드를 고정한 시험 입력, 기준선으로 이루어진다. 명세는 특정 라이브러리의 사용을 지시하지 않으며, 블록화 같은 기법은 허용되는 선택으로만 언급한다. 라이브러리에 위임할지 여부가 관찰 대상이기 때문이다. 자료형은 float32 또는 int32이며, 입력은 읽기 전용이고 출력 버퍼는 호출자가 할당한다.

기준선은 과제마다 하나로 고정한다. 다섯 과제는 연산 전체에 대응하는 NumPy 루틴을, 일곱 과제는 별도로 준비한 C+OpenMP 참조 구현을 기준선으로 삼았다. 기준선은 성능의 상한이 아니라 조건 간 비교를 위한 공통 기준점이며, 그 선택 규칙과 최적화 수준은 과제마다 고르지 않다(4.5절).

3.3 평가 하네스

하네스의 흐름은 그림 1과 같다. 프롬프트는 공통 지시문, 언어별 환경 설명, 과제 명세로 구성된다. 공통 지시문은 모델에 고성능 수치 프로그래밍 전문가의 역할을 부여하고, 정확성을 우선하되 가능한 한 빠른 구현을 작성하도록 요구한다. 언어별 환경 설명에는 컴파일 명령 전체, 링크되는 라이브러리, CPU의 코어 수·명령어 집합·캐시 크기, 고정된 스레드 수를 적어 모든 조건에 같은 항목의 정보를 공개하였다. 응답에서 코드 블록 하나를 추출하여 요구한 시그니처와 대조하고, C 구현은 모두 같은 플래그(`gcc -O3 -march=native`, `c_openmp`는 `-fopenmp` 추가)로 빌드한다. 이하에서 응답은 모델의 출력 180개를, 구현은 그중 코드가 추출된 176개를 가리킨다.

생성된 코드는 메모리 한도와 시간 제한을 건 별도 프로세스에서만 실행한다. 정확성은 크기가 다른 세 입력 형태에서 각각 두 번 호출한 결과를 배정밀도 참조 결과와 비교하여, 과제별 허용오차(상대 오차 10^{-5}~10^{-3}, 정수 출력은 완전 일치) 이내이면 통과로 판정한다. 출력 버퍼는 미리 감시값으로 채우고 앞뒤에 보호 구역을 두어 기록 누락, 범위 밖 쓰기, 입력 변경을 탐지하며, 두 호출의 결과가 서로 다르면 경쟁 상태로 판정한다. 세 형태를 모두 통과한 구현만 성능 측정 크기에서 시간을 측정한다.

실패는 실험 전에 정의한 9개 오류 유형 가운데 하나로 자동 분류한다. 예를 들어 출력이 모든 원소에서 허용오차의 일정 배수(기본 20배) 안으로만 어긋나면 `numeric_instability`, 출력을 전치하거나 차원을 맞바꾸었을 때 참조와 일치하면 `arg_order`로 분류한다. 분류 규칙은 결함을 의도적으로 심은 모의 응답으로 검증하였고, 하네스 전체에 276개의 단위 테스트를 두었다.

3.4 측정 규약

측정 규약은 사람이 지키는 수칙으로 두지 않고 하네스가 강제하며 결과 행에 기록한다. (1) 프로세스를 성능 코어 4개에 고정하고 스레드 수를 4로 고정한다(`c_seq`는 코어 1개). (2) CPU 주파수 거버너가 `performance`인지 확인한다. (3) 입력을 호출 전에 메모리에 올려 두고, 워밍업 3회 뒤 10회 호출의 중앙값을 쓰되 모든 호출 시간을 보존한다. (4) 생성 코드와 기준선을 같은 프로세스에서 번갈아 호출하고 쌍마다 순서를 바꾼다(이하 교차 측정). (5) 고정 코어의 사전 사용률이나 기준선 시간의 변동이 임계값을 넘으면 그 행에 규약 플래그를 남기고, 부하 평균과 온도, 동작 주파수를 함께 기록한다. 교차 측정을 둔 까닭은, 실험에 사용한 노트북에서 지속 부하 뒤 동작 주파수가 절반 수준으로 떨어지는 현상이 관찰되어 같은 시점에 측정한 두 시간의 비율만 안정적이었기 때문이다.

규약과 추출 규칙은 실험 도중 세 차례 바뀌었다. 부하 평균 플래그를 정보용으로 낮추었고, 코드 블록이 없는 응답의 판정 규칙을 고쳐 2건을 `compile_error`에서 `signature_mismatch`로 다시 분류하였으며, 교차 측정을 도입하였다. 변경 뒤에는 생성된 응답은 그대로 두고 모든 구현을 다시 분류하고 측정하였다. 오류 유형의 정의는 바꾸지 않았다.

표 2 조건별 첫 시도 통과 수(5개 응답 중)와 통과한 구현의 속도비 중앙값. 마지막 열은 `c_openmp`의 속도비 중앙값을 `numpy`의 것으로 나눈 값이다(반올림 전의 값으로 계산). ‡는 측정 결함(4.5절)의 영향을 받은 값으로, 재측정 뒤 갱신한다.

과제	numpy		c_seq		c_openmp		비율
	통과	속도비	통과	속도비	통과	속도비	
gemm	5/5	0.94	5/5	0.35	5/5	0.87	0.93
jacobi2d	5/5	0.22‡	5/5	0.71	4/5	1.45	6.64‡
spmv_csr	5/5	0.13‡	4/5	0.35	5/5	1.03	8.14‡
dot	5/5	1.44	5/5	1.03	5/5	1.67	1.15
histogram	5/5	0.19‡	5/5	0.54	5/5	1.52	8.04‡
scan	5/5	0.15‡	5/5	0.75	3/5	1.15	7.71‡
nbody	0/5	–	5/5	2.47	5/5	7.95	–
sort	5/5	1.10	5/5	0.69	4/5	1.28	1.16
cholesky	5/5	3.43	5/5	2.88	4/5	4.85	1.41
floyd_warshall	5/5	0.08‡	4/5	0.99	3/5	1.43	18.53‡
bfs	5/5	0.17	3/5	0.73	5/5	1.00	5.99
fft	5/5	1.05	4/5	2.90	5/5	6.65	6.35
합계	55/60 통과		55/60 통과		53/60 통과		

4. 중간 결과

4.1 실험 설정

모델은 Anthropic의 Claude Sonnet 5.5(`claude-sonnet-5-5`) 한 종이며 추론 강도는 medium으로 고정하였다. 이 모델은 temperature 매개변수를 받지 않으므로 제공자의 기본 샘플링을 사용하였다. 12개 과제 × 3개 언어 조건 × 5개 샘플 = 180개 응답을 2026년 11월 8~9일에 서로 독립적으로 생성하였고, 오류 메시지를 되돌려 주는 수정 루프는 사용하지 않았다. 따라서 이하의 통과는 모두 첫 시도 통과이다. 실험 머신은 Intel Core i7-1260P(성능 코어 4개, AVX2) 노트북이고 Ubuntu 26.04, GCC 15.2, Python 3.14, NumPy 2.3.5(OpenBLAS 0.3.30)를 사용하였다.

사용한 토큰은 입력 약 56.7만 개, 출력 약 35.1만 개(공시 단가 기준 약 4.6달러)이다. 추론 토큰을 제외한 응답당 평균 출력 토큰은 `numpy`가 약 970개, C의 두 조건이 약 1,900개였다. 추론 토큰은 `fft`의 C 응답 10개에서만 발생하였고 전체 출력 토큰의 18%를 차지한다. 프롬프트, 응답 원문, 결과 행은 프로젝트 저장소에 보존하였으며, 하네스 검증용 예비 실행(`gemm` 9회)은 집계에서 제외하였다.

4.2 정확성

표 2에 조건별 통과 수와 속도비의 중앙값을 정리하였다. 통과 수는 `numpy` 55/60, `c_seq` 55/60, `c_openmp` 53/60으로 비슷하며, 이 표본 크기에서 의미 있는 차이라고 할 수 없다. 코드 블록이 없는 응답을 제외하면 55/60, 55/59, 53/57이다. 난이도 수준별 통과 수는 수준 1이 58/60, 수준 2가 52/60, 수준 3이 53/60이었다. 컴파일 오류, 시간 초과, 인자 순서 오류, 경쟁 상태로 분류된 실패는 없었다.

통과 수와 달리 실패의 원인은 조건마다 달랐다(표 3). `numpy`의 실패 5건은 모두 `nbody`에서 라이브러리에 위임하기 위해 식을 재배열한 구현에서 나왔다(4.4절). C의 알고리즘·메모리 오류 8건은 모두 블록화, SIMD 누적 합, 방향 최적화, four-step 분해처럼 모델이 단순한 구현 대신 택한 최적화 구조에서 나왔다. `floyd_warshall`에서는 C 구현 10개가 모두 블록화

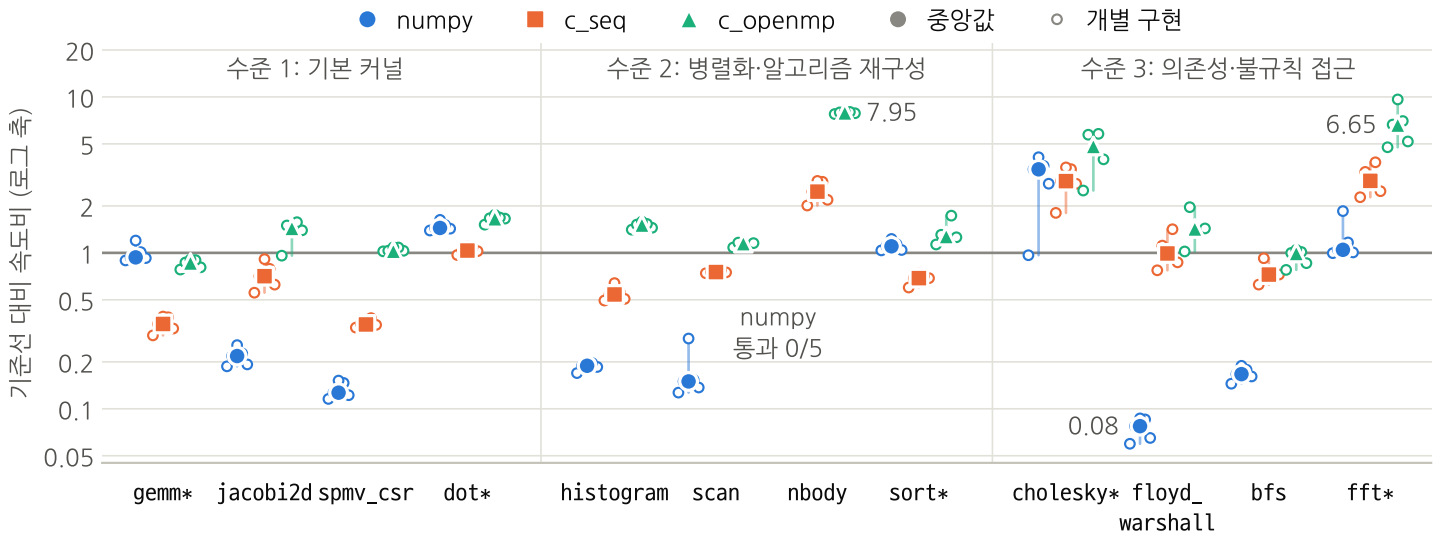


그림 2 통과한 구현의 기준선 대비 속도비(로그 축; 1보다 크면 기준선보다 빠르다). 큰 표식은 조건별 중앙값, 작은 점은 개별 구현, 세로선은 최소–최대 범위이다. *는 기준선이 NumPy 루틴인 과제이다. 기준선은 4스레드 환경에서 실행하지만 `dot`, `sort`, `fft`의 NumPy 루틴은 단일 스레드로 동작하며, `c_seq`는 단일 스레드이다. 칸마다 구현 수는 표 2의 통과 수와 같다.

표 3 실패 17건의 원인(생성 코드를 읽어 판정)과 조건별 건수. 분류는 하네스의 자동 분류이다(W: `wrong_output`, N: `numeric_instability`, R: `runtime_crash`, S: `signature_mismatch`).

원인(과제)	분류	numpy	c_seq	c_openmp
위임을 위한 식 재배열의 float32 상쇄 (`nbody`)	N 3, W 2	5	–	–
블록화 구현의 루프 순서 오류 (`floyd_warshall`)	W 3	–	1	2
블록·SIMD 구현의 색인·경계 오류 (`scan`, `cholesky`)	W 2	–	–	2
방향 그래프에 bottom-up 탐색 적용 (`bfs`)	W 2	–	2	–
힙 메모리 오류로 실행 중단 (`fft`)	R 1	–	1	–
코드 블록이 없는 응답 (`jacobi2d`, `spmv_csr`, `scan`, `sort`)	S 4	–	1	3
합계		5	5	7

알고리즘이었고, 그중 3개가 블록 안의 루프 순서를 잘못 두어 틀린 결과를 출력하였다. `bfs`에서는 C 응답 10개가 모두 방향 최적화(direction-optimizing) 탐색을 먼저 제안하였다. 그중 8개는 주어진 그래프가 방향 그래프이므로 bottom-up 단계에 역방향 간선이 필요하다는 점을 응답의 설명에서 스스로 밝히고 top-down 탐색만 구현하여 통과하였다. bottom-up 단계를 남겨 둔 나머지 2개는 잘못된 거리를 출력하였다. 다만 C 구현의 대부분이 이러한 최적화 구조를 택하였으므로, 단순한 구현과 비교한 실패율은 이 자료로 알 수 없다. 형식 위반 4건은 코드 블록 없이 응답한 경우이며, 요구한 시그니처를 어긴 구현은 없었다.

4.3 성능

그림 2는 통과한 구현의 속도비 분포를 나타낸다. 기준선의 최적화 수준이 과제마다 다르므로(4.5절), 먼저 같은 과제 안에서 두 조건의 속도비 중앙값의 비를 본다(표 2의 마지막 열). 측정의 분해능은 `gemm`의 `numpy` 조건에서 가늠할 수 있다. 이 구현은 기준선과 같은 행렬 곱 호출이므로 속도비가 1이어야 하지만 실제로는 0.89~1.20(중앙값 0.94)이었다. 따라서 이 환경에서 20% 이내의 차이는 해석하지 않는다.

`c_openmp` 구현의 속도는 `numpy` 구현의 0.93배(`gemm`), 1.15배(`dot`), 1.16배(`sort`), 1.41배(`cholesky`)였다. 이 네 과제에서 두 조건의 차이는 분해능 안이거나 1.5배 이내이다. 넷은 모두 `numpy` 구현이 연산의 대부분을 고도로 최적화된 라이브러리 루틴(BLAS의 행렬 곱과 내적, SIMD 정렬)에 위임한 과제이다. 반면 `nbody`를 제외한 나머지 일곱 과제에서는 `c_openmp`가 6.0~18.5배 빨랐다. 이 일곱 과제는 연산 전체를 위임할 루틴이 없거나(`jacobi2d`, `spmv_csr`, `floyd_warshall`, `bfs`), 위임할 루틴이 있어도 그 루틴이 4스레드 C 구현보다 느린 과제이다(`histogram`, `scan`, `fft`).

이 결과는 두 예측과 모순되지 않는다. 그러나 가설을 지지한다고 말하기에는 이르다. 과제를 두 그룹으로 나눈 기준은 결과를 확인한 뒤에 세운 사후 설명이고, 지금의 양 끝 조건에서는 추상화 수준과 위임 가능성이 함께 변하므로 두 설명을 구분할 수 없다. 한편 모든 결정을 직접 내려야 하는 `c_openmp` 조건에서 속도 손실은 관찰되지 않았다(모든 과제에서 속도비 0.87 이상). 직접 결정의 비용은 속도가 아니라 정확성(4.2절의 C 실패)에서 나타났다.

`numpy` 조건의 속도는 대체로 위임 대상 루틴의 품질에 묶여 있었으나, 모델이 위임의 방식을 직접 정한 경우에는 결과가 달라졌다. `cholesky`에서는 블록 단위 분해에 행렬 곱을 조합한 구현 4개가 라이브러리의 분해 루틴을 한 번 호출하는 기준선보다 2.8~4.1배 빨랐고(같은 루틴을 한 번 호출한 나머지 1개는 0.97배), `dot`에서는 배열을 스레드로 나누어 라이브러리 내적을 호출한 구현이 기준선의 1.4~1.6배였다.

C의 두 조건을 비교하면 `c_openmp`의 속도비 중앙값은 모든 과제에서 `c_seq`보다 높았다(1.4~3.2배). 다만 `cholesky`, `floyd_warshall`, `bfs`에서는 일부 OpenMP 구현이 일부 순차 구현보다 느렸다. 같은 칸의 구현은 서로 다른 프로그램이므

로 이 역전에는 코드 품질의 차이와 측정 잡음이 섞여 있으며, 측정 장비를 옮겨 같은 코드를 반복 측정하면 둘을 가를 수 있다(5장).

기준선 대비 속도비로 보면 numpy는 기준선이 NumPy 루틴인 다섯 과제에서 0.94~3.43이었고, 기준선이 C+OpenMP 구현인 과제 가운데 numpy 구현이 통과한 여섯 과제에서 0.08~0.22이었다. 같은 여섯 과제에서 c_openmp는 1.00~1.52, c_seq는 0.35~0.99이었다. 속도비가 1을 크게 넘는 칸(nbody, cholesky, fft의 C 조건 2.47~7.95, cholesky의 numpy 조건 3.43)은 생성 코드가 빠르기 때문이기도 하지만 해당 기준선의 최적화 수준이 낮기 때문이기도 하다.

4.4 생성 코드에 나타난 결정

성능 결정을 실제로 누가 내렸는지 확인하기 위해 추출된 구현의 소스를 패턴 검색과 직접 판독으로 조사하였다.

위임. numpy 조건의 gemm 구현 5개는 모두 행렬 곱 호출 하나였고, cholesky는 5개 중 4개가 블록 단위 분해에 행렬 곱을 조합한 구조였다.

NumPy 조건에서의 직접 병렬화. 위임할 병렬 루틴이 없는 과제에서는 numpy 조건에서도 모델이 병렬화를 직접 구현하였다. numpy 구현 60개 중 32개가 Python 스레드로 배열을 나누어 처리하였고, jacobi2d, dot, histogram, scan, nbody에서는 5개 모두가 그러하였다. 이 방식은 dot에서는 효과가 있었으나 jacobi2d, histogram, scan에서는 c_openmp보다 6.6~8.0배 느렸다(이 세 과제의 수치는 4.5절의 측정 결함의 영향을 받았다).

C 조건에서의 직접 결정. C의 두 조건에서 모델은 저수준 결정을 직접 내렸다. 코드가 추출된 C 구현 116개 중 64개가 SIMD 내장 함수(AVX2·FMA)를 직접 사용하였다. gemm은 10개 모두 캐시 블록화와 패킹을 갖춘 구조였고 [7], nbody는 10개 모두 좌표를 성분별 배열(SoA)로 옮기고 근사 역제곱근 명령을 사용하였다. 4.2절의 실패 사례에서 보듯, 이러한 적극적 최적화에는 정확성을 잃을 위험이 따른다.

위임을 위한 변형의 수치적 부작용. nbody의 numpy 구현 5개는 모두 가속도의 합 $\sum_j w_{ij}(\boldsymbol{x}_j - \boldsymbol{x}_i)$를 $\sum_j w_{ij}\boldsymbol{x}_j - \boldsymbol{x}_i \sum_j w_{ij}$ 꼴로 재배열하여 행렬 곱으로 계산하였다. 여기서 $w_{ij} = m_j / \left(\|\boldsymbol{x}_j - \boldsymbol{x}_i\|^2 + \varepsilon^2 \right)^{3/2}$이고 $\varepsilon^2 = 10^{-4}$이다. 이 꼴에서는 매우 큰 자기 항 w_{ii}가 두 항에 함께 들어가 float32에서 상쇄 오차가 남는다. 그 결과 5개 모두 작은 두 입력 형태($n = 5, 301$)에서 허용오차(상대 오차 10^{-3}과 참조 결과 RMS의 10^{-3}배)를 1.1~20배 넘었다. 같은 과제의 C 구현 10개는 좌표의 차를 쌍마다 직접 계산하여 모두 통과하였다. 위임을 위한 변형이 수치적 위험을 동반할 수 있음을 보이는 사례이나, 한 과제의 관찰이며 허용오차 설계에 민감하다. 명세가 전체 쌍의 표는 메모리에 들어가지 않는다고 알린 점도 이 변형을 유도하였을 수 있다. 한편 설계 단계에서 수치적 함정으로 둔 dot의 누적 정밀도와 fft의 회전 인자 정밀도에서는 실패가 없었다. dot의 C 구현 10개는 여러 개의 누산기로 나누어 더하였고, fft의 C 구현 10개는 회전 인자를 배정밀도로 계산하였다.

4.5 타당성 위협

표본. 모델 1종, 언어 조건 3개, 과제·조건 조합당 5개 샘플의 예비 실험이다. 통과 수의 차이에 통계적 의미를 부여할 수 없고, 속도비는 통과한 구현만으로 계산하므로 통과한 구현이 3개뿐인 칸의 중앙값은 다른 칸과 직접 비교하기 어렵다. 다른 모델에서도 같은 양상이 나타나는지는 확인하지 않았다.

기준선. 기준선의 최적화 수준과 스레드 사용이 과제마다 다르다. dot, sort, fft의 기준선인 NumPy 루틴은 단일 스레드로 동작하며, histogram과 scan은 대응하는 NumPy 루틴이 있음에도 C+OpenMP 구현을 기준선으로 삼아 선택 규칙이 일관되지 않다. 따라서 기준선 대비 속도비를 과제 사이에서 직접 비교할 수 없고, c_seq의 속도비에는 기준선과의 스레드 수 차이가 그대로 포함된다. 또한 C+OpenMP 기준선과 참조 구현은 AI 코딩 도구의 지원을 받아 작성하였으므로, c_openmp 구현이 기준선과 비슷하다는 사실은 독립적인 성능 기준이 되지 못한다.

측정. 노트북에서 측정하여 절대 시간의 변동이 크다. 시간을 측정한 163개 구현에서 호출 간 변동계수의 중앙값은 7.4%이고, 48개(29%)에 규약 플래그가 붙었으며(그중 47건은 기준선 시간의 변동) 표 2와 그림 2에는 이들도 포함하였다. 또한 이 초안을 검토하는 과정에서 교차 측정의 결함을 발견하였다. 기준선이 OpenMP 구현인 과제에서는 OpenMP 런타임이 프로세스의 주 스레드를 코어 하나에 고정하는데, 그 뒤에 호출된 numpy 구현이 만든 스레드가 이 고정을 물려받아 한 코어에서 실행되었다. 스레드를 사용한 numpy 구현이 있는 다섯 과제(floyd_warshall, histogram, jacobi2d, scan, spmv_csr)의 numpy 측정값이 이 영향을 받아 실제보다 느리게 측정되었다. 하네스는 수정하였으며 전체를 다시 측정할 예정이다. 최종 수치는 데스크톱 또는 서버에서 다시 측정해야 한다.

프롬프트. 모든 응답은 정확한 구현 가운데 가장 빠른 것을 요구하고 하드웨어와 스레드 수를 공개한 프롬프트 하나에서 얻었다. 4.4절의 적극적 최적화와 그에 따른 실패는 이 지시의 영향을 받았을 수 있다. 또한 temperature를 고정할 수 없어 제공자의 기본 샘플링에 의존하였다.

과제와 분석 방법. 12개 과제는 교과서와 공개 벤치마크에 실린 커널이므로, C 조건의 높은 속도는 모델이 학습 과정에서 접한 구현을 재현한 결과일 수 있다. 정확성은 세 입력 형태로만 검사하였다. 4.4절의 집계와 표 3의 원인 판정은 패턴 검색과 소스 판독에 따른 것이며, 성능 결정을 자동으로 추출하는 도구는 아직 구현하지 않았다.

5. 결론 및 향후 계획

본 중간보고서에서는 LLM이 생성한 HPC 코드를 언어 조건별로 비교하기 위한 평가 하네스를 구현하고, 12개 과제와 세 언어 조건에 대한 예비 결과를 얻었다. 첫째, 통과 수(첫 시도 기준)는 조건 간에 비슷하였으나 실패의 원인은 조건마다 달랐다. 둘째, 같은 과제 안에서 numpy와 c_openmp의 속도는 numpy 구현이 최적화된 라이브러리 루틴에 위임한 네 과제에서 비슷하였고 나머지 일곱 과제에서 크게 벌어졌다. 이는 가설의 두 예측과 모순되지 않으나, 사후에 세운 구분이며 추상화 수준과 위임 가능성이 함께 변하는 조건만을 비교한 결과

이다. 셋째, 위임을 위한 식 재배열이 수치 오류로 이어진 사례가 한 과제에서 관찰되었다.

남은 계획은 표 4와 같다. 11월 11일 면담에서 측정을 리눅스 데스크톱으로 옮기고 반복 횟수를 늘려 평균을 내며, Halide처럼 CPU를 대상으로 하는 최근 고성능 언어를 추가하기로 하였다. 이에 따라 `numba`, `halide_auto`, `halide_manual`, `rust`(+rayon), `mojo`를 추가하고, 모든 조건의 코드를 최종 장비에서 다시 생성하여 서로 다른 시각에 3회 반복한 측정(총 30회)의 평균과 95% 신뢰구간으로 비교한다. 위임할 루틴이 없는 과제에서 `numba`(반복문을 직접 작성)와 `halide_auto`(스케줄을 자동 탐색에 위임)의 속도가 `numpy`에 머무는지 C에 가까워지는지가 두 설명을 가른다. 과제의 위임 가능성은 생성 결과와 무관한 사전 기준(라이브러리 호출 한 번의 속도)으로 정의하고, 1차 실험의 코드도 새 장비에서 다시 측정하여 측정 환경의 영향을 확인한다. 이후의 연구 방향으로는 LLM 기반 최적화 패스 순서 결정(phase ordering)이 언어와 컴파일러에 따라 달라지는지를 검토한다.

표 4 남은 연구 계획(수행 순서)

단계	내용	확인하려는 것	구분
1. 측정 환경 이전	리눅스 데스크톱으로 이전, 서로 다른 시각의 3회 × 10회 측정의 평균·95% 신뢰구간, 1차 실험 코드 재측정	측정 환경의 영향(면담 지적)	필수
2. 언어 조건 확장	`numba`, `halide_auto`·`halide_manual`, `rust`(+rayon), `mojo` 추가	결정을 직접 표현하거나 자동 탐색에 위임할 때의 격차 변화(P1, P2)	필수
3. 재생성과 재측정	모든 조건을 최종 장비에서 다시 생성(12과제 × 8조건 × 5샘플)·측정	같은 프롬프트·장비에서의 공정한 비교	필수
4. 수정 루프와 세부 조건	오류 메시지를 되돌려 주는 최대 3턴의 수정 루프, 피드백 종류(컴파일러 / 컴파일러+린트), 인자 전달 방식, 문법 안내의 유무	첫 시도 실패의 회복 가능성	필수
5. 안전성과 결정 요약	ASan·UBSan 재실행, 린트 연결, 컴파일러의 벡터화 보고와 소스 분석에 따른 성능 결정 요약의 자동 추출	메모리 오류의 판정, 소스 판독의 대체	필수
6. 분석과 보고	위임 가능성의 사전 기준, 기준선 정책 재검토, pass@k와 부트스트랩 신뢰구간, 최종보고서	가설의 두 예측	필수
7. 확장	두 번째 모델, 프롬프트 조건, 후속 연구로 LLM 기반 phase ordering	모델·프롬프트·컴파일러에 대한 일반화	선택

참 고 문 헌

[1] M. Chen et al., "Evaluating Large Language Models Trained on Code," *arXiv preprint arXiv:2107.03374*, 2021.

[2] F. Cassano et al., "MultiPL-E: A Scalable and Polyglot Approach to Benchmarking Neural Code Generation," *IEEE Transactions on Software Engineering*, vol. 49, no. 7, pp. 3675–3691, 2023.

[3] D. Nichols, J. H. Davis, Z. Xie, A. Rajaram, and A. Bhatele, "Can Large Language Models Write Parallel Code?," *Proc. 33rd International Symposium on High-Performance Parallel and Distributed Computing (HPDC)*, pp. 281–294, 2024.

[4] W. F. Godoy, P. Valero-Lara, K. Teranishi, P. Balaprakash, and J. S. Vetter, "Evaluation of OpenAI Codex for HPC Parallel Programming Models Kernel Generation," *Proc. 52nd International Conference on Parallel Processing Workshops (ICPP Workshops)*, pp. 136–144, 2023.

[5] P. Valero-Lara et al., "Comparing Llama-2 and GPT-3 LLMs for HPC Kernels Generation," *Languages and Compilers for Parallel Computing (LCPC 2023), Lecture Notes in Computer Science*, vol. 14480, pp. 20–32, 2026.

[6] P. Diehl, N. Nader, S. Brandt, and H. Kaiser, "Evaluating AI-Generated Code for C++, Fortran, Go, Java, Julia, Matlab, Python, R, and Rust," *Euro-Par 2024: Parallel Processing Workshops, Lecture Notes in Computer Science*, vol. 15385, pp. 243–254, 2025.

[7] K. Goto and R. A. van de Geijn, "Anatomy of High-Performance Matrix Multiplication," *ACM Transactions on Mathematical Software*, vol. 34, no. 3, pp. 12:1–12:25, 2008.

[8] C. R. Harris et al., "Array Programming with NumPy," *Nature*, vol. 585, no. 7825, pp. 357–362, 2020.

[9] J. Ragan-Kelley, C. Barnes, A. Adams, S. Paris, F. Durand, and S. Amarasinghe, "Halide: A Language and Compiler for Optimizing Parallelism, Locality, and Recomputation in Image Processing Pipelines," *Proc. 34th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pp. 519–530, 2013.

[10] S. K. Lam, A. Pitrou, and S. Seibert, "Numba: A LLVM-based Python JIT Compiler," *Proc. 2nd Workshop on the LLVM Compiler Infrastructure in HPC (LLVM '15)*, pp. 7:1–7:6, 2015.

[11] A. Shypula et al., "Learning Performance-Improving Code Edits," *Proc. 12th International Conference on Learning Representations (ICLR)*, 2024.

[12] Y. Qing et al., "EffiBench-X: A Multi-Language Benchmark for Measuring Efficiency of LLM-Generated Code," *Advances in Neural Information Processing Systems 38 (NeurIPS), Datasets and Benchmarks Track*, pp. 82577–82620, 2025.

[13] H. Pearce, B. Ahmad, B. Tan, B. Dolan-Gavitt, and R. Karri, "Asleep at the Keyboard? Assessing the Security of GitHub Copilot's Code Contributions," *Proc. 2022 IEEE Symposium on Security and Privacy (SP)*, pp. 754–768, 2022.

[14] L.-N. Pouchet, "PolyBench/C: The Polyhedral Benchmark Suite," https://www.cs.colostate.edu/~pouchet/software/polybench/, 2012.

[15] D. H. Bailey et al., "The NAS Parallel Benchmarks," *The International Journal of Supercomputer Applications*, vol. 5, no. 3, pp. 63–73, 1991.

[16] S. Beamer, K. Asanović, and D. Patterson, "The GAP Benchmark Suite," *arXiv preprint arXiv:1508.03619*, 2015.